

Chapter #9:

Logical Design

- Logical design is a set of steps to create the structure of a data mart.
- It starts with a conceptual design outlining the application domain.
- The logical design then defines the data structures that will represent the data mart according to the chosen logical model.
- Also optimizes performance by fine-tuning these structures.

Steps in transforming Conceptual schema into logical schema:

1. Translating fact schemata into logical schemata:

This involves connecting the fact schemata into star, snowflake, and constellation schemata.

Materializing views:

- This involves creating materialized views, which are pre-computed results or queries that can be stored and used to improve query performance.

Fragmenting fact tables:

- This involves breaking down fact tables into smaller, more manageable pieces, either vertically or horizontally.

From Fact Schemata to fact Star Schemata:

- Fact schemata can be directly translated into star schemata.
- However, this translation is not fully automatic and requires careful design considerations.
- A relational implementation of a fact schema generally involves a star schema with a fact table containing all measures and descriptive attributes linked to the fact.

Descriptive Attributes:

- Descriptive attributes are pieces of information that cannot be used for aggregation but are still useful.
- If descriptive attribute is directly linked to a fact, it should be included in fact table.
- If linked with dimensional attribute, it must be included in dimensional table.

Cross-Dimensional attributes:

- Cross-dimensional attributes define a many-to-many relationship between two or more dimensional attributes belonging to different hierarchies.
- Create a new table to represent this relationship.
- Include the cross-dimensional attribute and dimensional attributes as the PK.
- Using surrogate key can be a valid solution.

Shared Hierarchies:

- Sometimes a fact schema has repeating hierarchies or parts of hierarchies.
- In this case, avoid creating multiple dimension tables with similar data.
- There are two ways for dealing with shared hierarchies.

Total Sharding:

- Two hierarchies have exactly the same attributes with different meanings.
- Need to create only one dimension table for both hierarchies.
- The fact table will have two foreign keys referencing the same dimension table.

Partial Sharding:

- Two hierarchies share some of their attributes.
- Can be done by introducing redundancy by including shared attributes in both dimension tables or can be by creating a separate shared table for both common attributes.

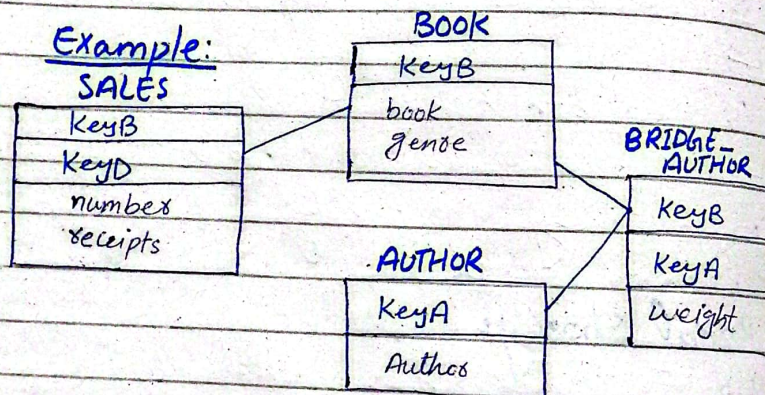
Multiple Aocs:

→ Sometimes hierarchies might show many-to-many relationships between entities.

→ One way to model this s/s by using a bridge table.

→ A "bridge table" contains a combination of the attributes linked by the multiple Aoc.

Example:



→ Using the bridge table, two types of queries can be performed:

- Weighted Query
- Impact Query

→ Sometimes, push down the multiple Aocs to create a new dimension. This is done when the end attribute of the multiple

aoc has child attributes that form a hierarchy.

Push-down

- Introduces redundancy in the fact tables.
- Fact table lines are repeated as many times.

vs Bridge-Table

- more concise fact tables.
- No redundancy in the fact tables.
- requires a bridge table.

Handling Optional Aocs:

⇒ Sometimes, hierarchy might have incomplete solutions.

NULL values

→ Use NULL values to represent missing values in optional hierarchies.

Fake values:

→ Considers using fake values to represent missing data more explicitly.

Handling incomplete Hierarchies:

⇒ Can be handled by:

- Upward balancing
- Downward Balancing
- Downward Balancing

Balancing by exclusion:

- Replacing missing values with a generic signpost like "other".

Downward Balancing:

- Replace missing values with the value of the preceding attribute in the hierarchy.

Upward Balancing:

- Replace missing values with the value of the following attribute in the hierarchy.

Modeling Recursive Hierarchies:

⇒ Recursive hierarchies are hierarchies where the number of levels can vary.

⇒ This makes them challenging to model in a traditional data warehouse.

⇒ There are two main ways to model these:

- Self-referencing FK
- Navigation Table.

Degenerate Dimensions:

⇒ A degenerate dimension is a hierarchy that consists of only one attribute.

⇒ There can be handles:

- By creating a separate dimension table.

- Include the degenerate dimensions directly in the fact tables.

Junk Dimensions:

⇒ A junk dimension is a dimension table that contains a set of degenerate dimensions.

⇒ This can help reduce the number of FK in the fact table.

⇒ Improve query performance

⇒ Better data granularity

Additivity issues:

⇒ When creating a DWH, it is important to consider how data will be aggregated.

⇒ Different operators have different properties.

* Aggregation operators:

- SUM, AVG, MIN, MAX

* Distributive operators:

- SUM & COUNT

* Algebraic Operators:

- MIN / MAX & AVG

* Holistic Operators:

- PERCENTILE & RANK, MODE & MEDIAN etc.

Snowflake Schemas

⇒ A snowflake schema involves normalizing dimension tables to reduce redundancy.

⇒ When snowflaking might be beneficial:

- Sparse Hierarchies
- High Cardinality Ratios
- Dynamic Hierarchies
- Readability

View Materialization

→ Data warehouse store large amount of data.

→ Smaller, more specific views of data are created

→ View materialization: is the process of selecting which of these views to make permanent.

→ Data in these views is pre-calculated and stored, so, it can access it quickly without having to recalculate it.

Why is it important?

- Faster query performance for specific questions
- Reduce load on data warehouse

How to choose views to materialize?

→ There are two main factors to consider:

Workload evaluation:

↳ Analyze how frequently certain views are used.

View maintenance cost:

↳ Materialized views need to be updated regularly to reflect changes in the underlying data.

Constraints on View Materialization

1. System Constraints:

◦ Disk Space:

- The amount of disk space available limits how many views can materialize.
- Need to estimate how much space each view will take up.

◦ Update Time:

- Updating materialized views takes time and impact other operations on the data warehouse.
- Need to consider how often views can be updated and how long it will take.

2. User Constraints:

◦ Query Response Time:

- Users expect quick answers to their queries.

- This limits the complexity of the materialized views.

◦ Data Freshness:

- Users want the data in the view to be up-to-date. Views should be updated frequently.

Challenges and Solutions: related to view Materialization:

Problems

- View materialization is a complex problem because there are many possible views that could be created, and each of them has its own costs and benefits.

- Need to balance the need for fast query performance with the limitations of system resources.

Solutions:

1. Transforming Constraints:

Simplify the problem by converting update time constraints into space constraints.

2. Multidimensional Lattice:

- This is a data structure that represents all the possible ways to group and aggregate data in a fact schema.

- It helps visualize the relationships between different views.

3. Candidate Views:

These are the views that are most useful for answering specific queries.

- They are selected based on their ability to answer multiple queries and their cost.

4. Materialization Algorithms:

- This algorithm helps to identify the best views to materialize.
- It considers the workload and the available resources.

View VS Materialized View

◦ Does not require space.	◦ Require space to store permanent.
◦ Provide security by restricting access to data.	◦ Provide snapshot of view.
◦ set of stored query.	⇒ INSERT DELETE UPDATE
◦ looks like a table but not table.	are not allowed on materialized view

View Fragmentation

View fragmentation is a technique used in data warehouse to improve system performance.

→ It involves breaking down a large table into smaller, more manageable fragments.

→ This can be done in two ways:

- Horizontal
- Vertical

Why is useful?

- Improved query performance
- Optimized storage.
- Parallel processing

Vertical view fragmentation

→ multicubing

⇒ This involves dividing a table into multiple parts, each containing a subset of columns but all the rows.

Characteristics:

- Consistency
- Completeness
- Non-redundancy

Reasons for vertical fragmentation:

- o Optimized workload cost
- o Saved Space
- o Reduced Key replication
- o Generalization

Horizontal Fragmentation

⇒ This involves dividing a table into a subset of rows but all the columns.

⇒ This helps to optimize storage and query performance.

⇒ It deals with individual cubes.

⇒ It does not require additional cost or disk space.

View materialization Diagram

